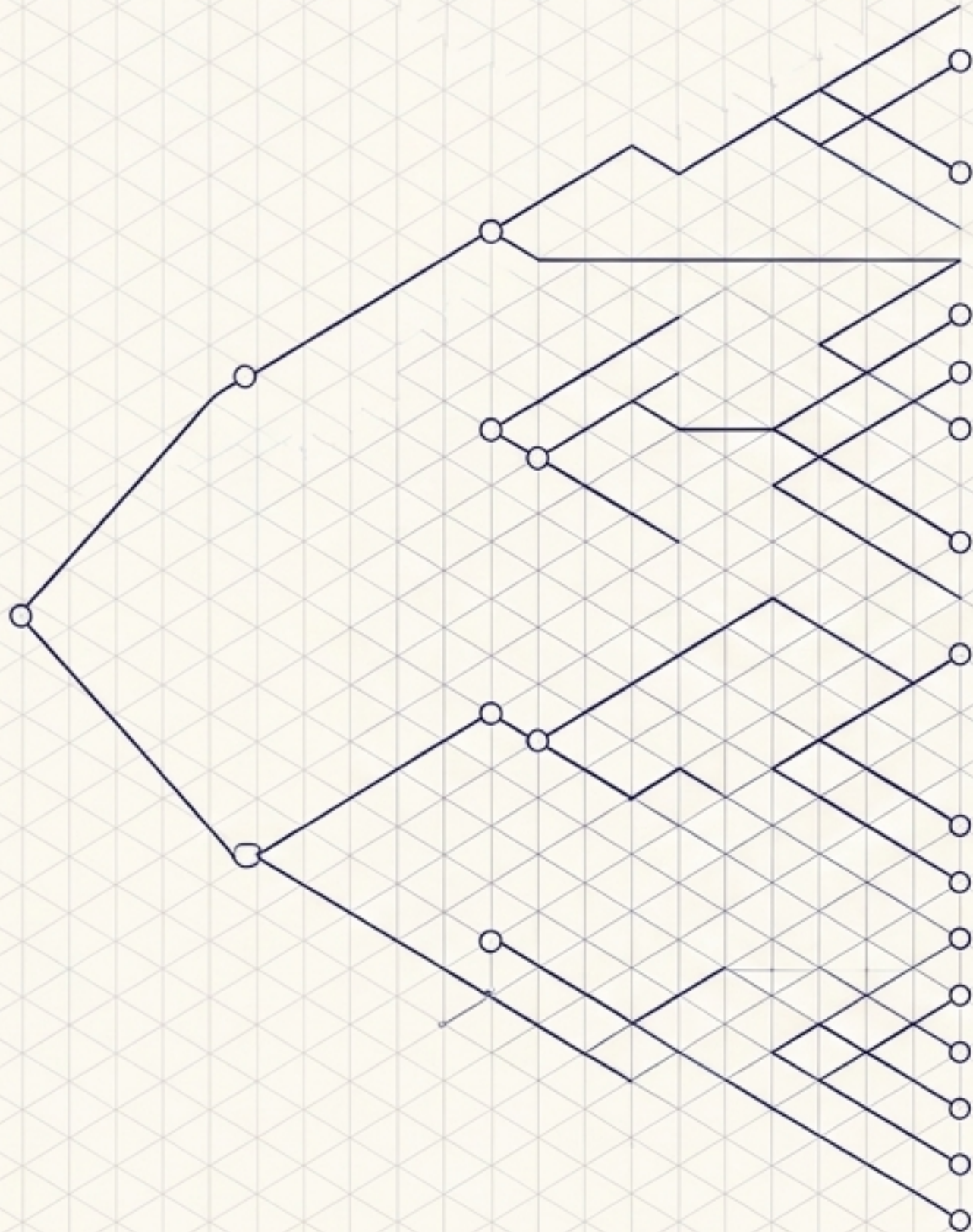




Deep Reinforcement Learning

Navigating Dynamic Environments through Action, Observation, and Reward

Based on MIT Introduction to Deep Learning
(Alexander Amini)

Classes of Learning Problems



Supervised Learning

- **Data:** (x, y) — Data + Label.
- **Goal:** Map x to y .
- **Agent's thought:** This thing is an apple.



Unsupervised Learning

- **Data:** x — Data only, no labels.
- **Goal:** Learn underlying structure.
- **Agent's thought:** This thing is like the other thing.



Reinforcement Learning

- **Data:** State-action pairs.
- **Goal:** Maximise future rewards over many time steps.
- **Agent's thought:** Eat this thing because it will keep you alive.

Reinforcement Learning (RL): Key Concepts



Reinforcement Learning (RL): Discounted Return

Total Return

The ultimate objective the agent wants to maximise from time t .

Discount Factor

A value between $0 < \gamma < 1$. It forces the agent to care slightly less about distant future rewards than immediate ones.

$$R_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

Immediate Reward

The payoff for the very next action.

Future Rewards

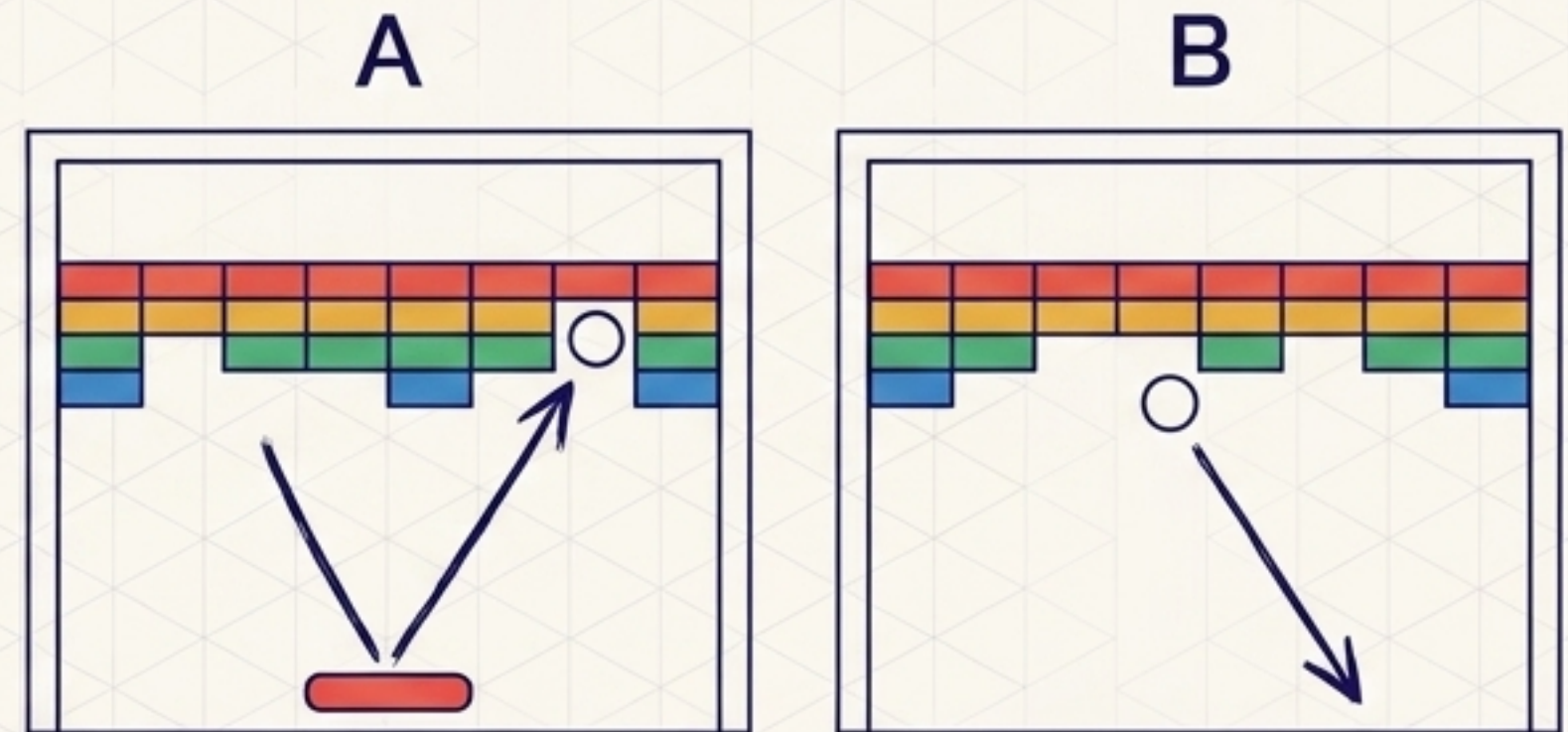
The downstream consequences of the agent's actions, heavily discounted by γ .

Defining the Q-Function

$$Q(s_t, a_t) = \mathbb{E}[R_t | s_t, a_t]$$

The expected total future reward an agent in a specific state (s) can receive by executing a certain action (a).

The Visual Proof: Atari Breakout



*Which (s, a) pair has
a higher Q-value?*

Because it is difficult for humans to accurately compute these values across millions of states, we use neural networks to estimate them.

Deep Reinforcement Learning



Value Learning

Find $Q(s, a)$

The agent learns the expected value of every possible action, then deterministically chooses the highest one.

$$a = \operatorname{argmax}_a Q(s, a)$$



Policy Learning

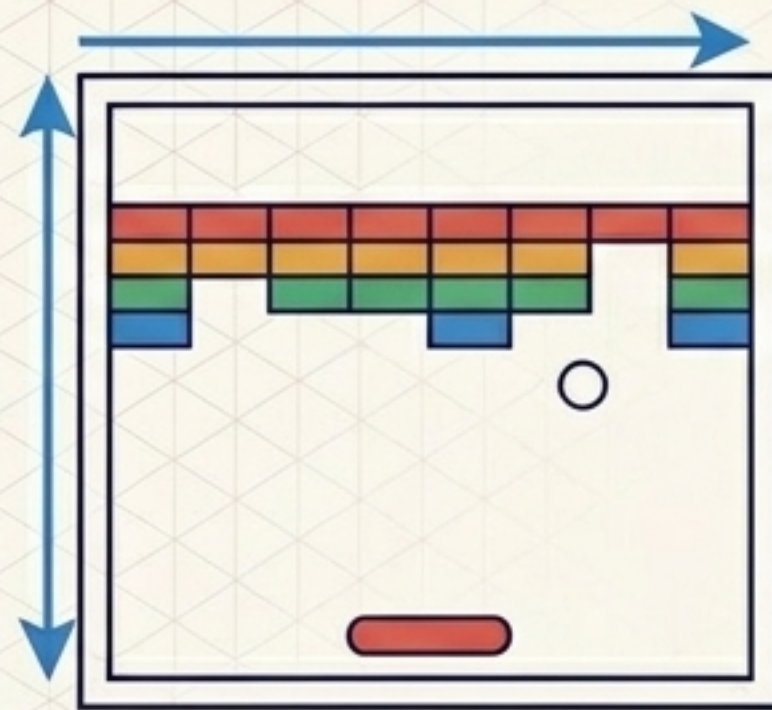
Find $\pi(s)$

The agent directly learns the policy—the probability distribution of the best actions to take—and samples from it.

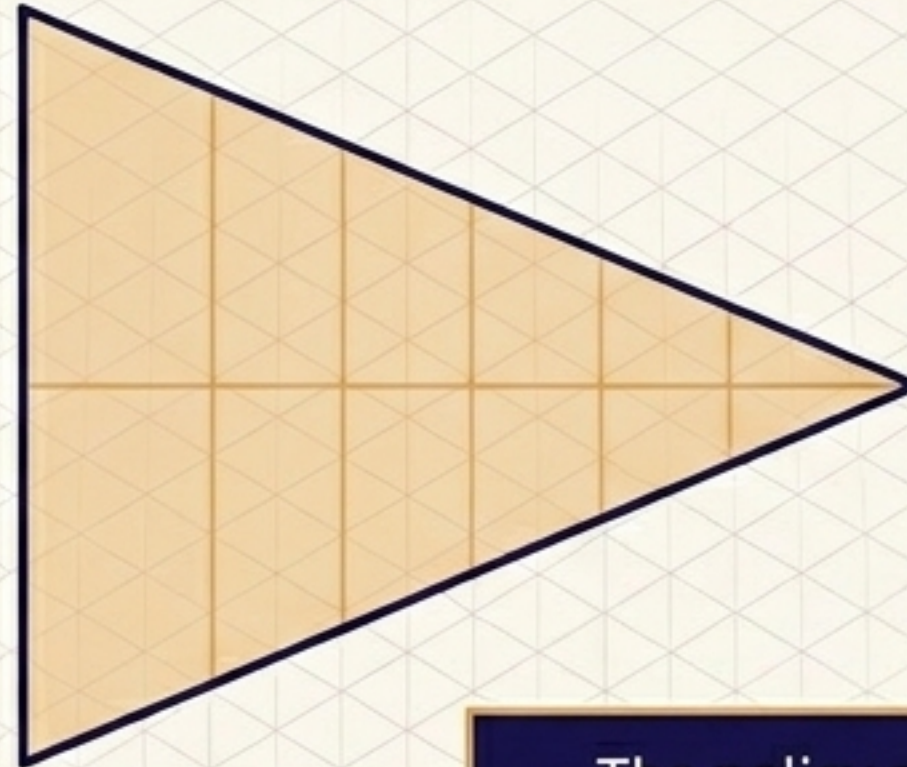
$$\text{Sample } a \sim \pi(s)$$

Deep Q Network Architecture

Input Layer:
State (s)



The Brain:
Deep Neural Network



Output Layer:
Action Values

$$Q(s, \text{left}) = 20$$

$$Q(s, \text{stay}) = 3$$

$$Q(s, \text{right}) = 0$$

The policy $\pi(s)$ is inferred by taking the argmax (the action with the highest estimated return).

Training DQN: Minimising the Error

$$\mathcal{L} = \mathbb{E} \left[\left\| \underbrace{\left(r + \gamma \max_{a'} Q(s', a') \right)}_{\text{Target Return}} - \underbrace{Q(s, a)}_{\text{Predicted Return}} \right\|^2 \right]$$

Target Return

The “ground truth” estimate. The immediate reward received, plus the best possible expected value for the very next step.

Predicted Return

The network’s initial prediction of the value before taking the action.

Training a Deep Q-Network is simply minimising the mathematical gap between what the agent expected to happen and what actually happened.

Discrete Action Space



Question: Which direction should I move? (Left, Stay, Right)

Verdict: Perfect for Q-Learning. An agent can easily calculate and compare distinct values for a small, finite number of choices.

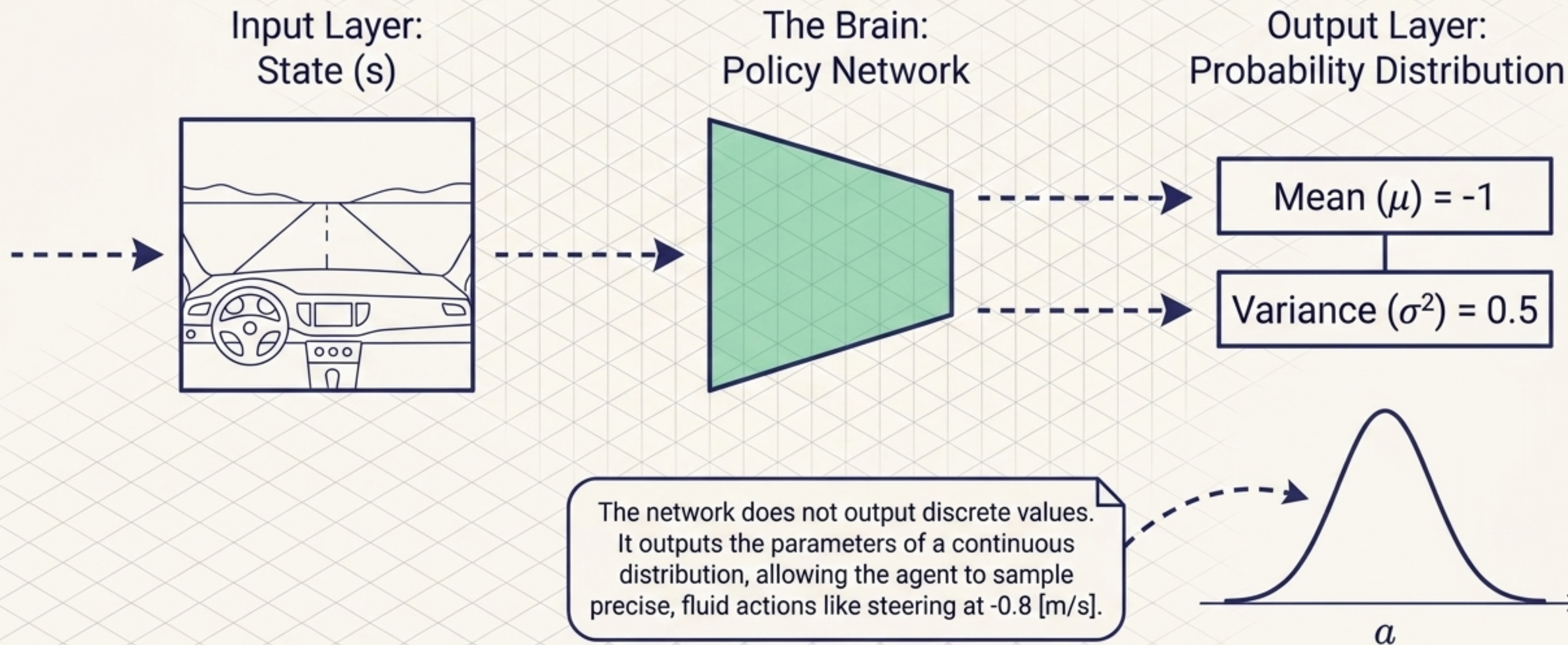
Continuous Action Space



Question: At what precise angle and speed should I steer? (e.g., 0.71 m/s vs 0.72 m/s)

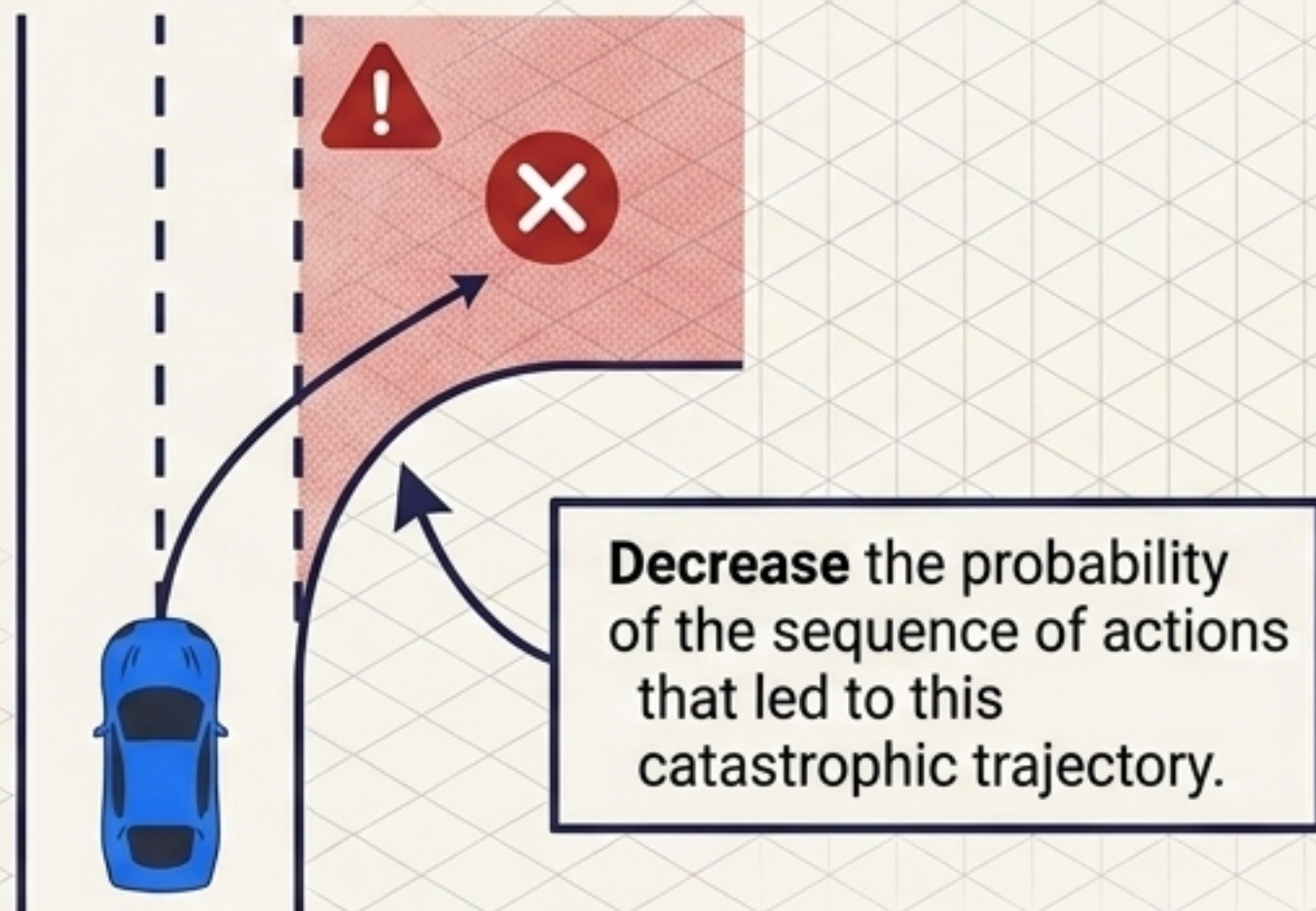
Verdict: Q-Learning fails. It is mathematically impossible to assign a distinct Q-value to an infinite number of decimal points and angles.

Policy Gradient (PG): Architecture

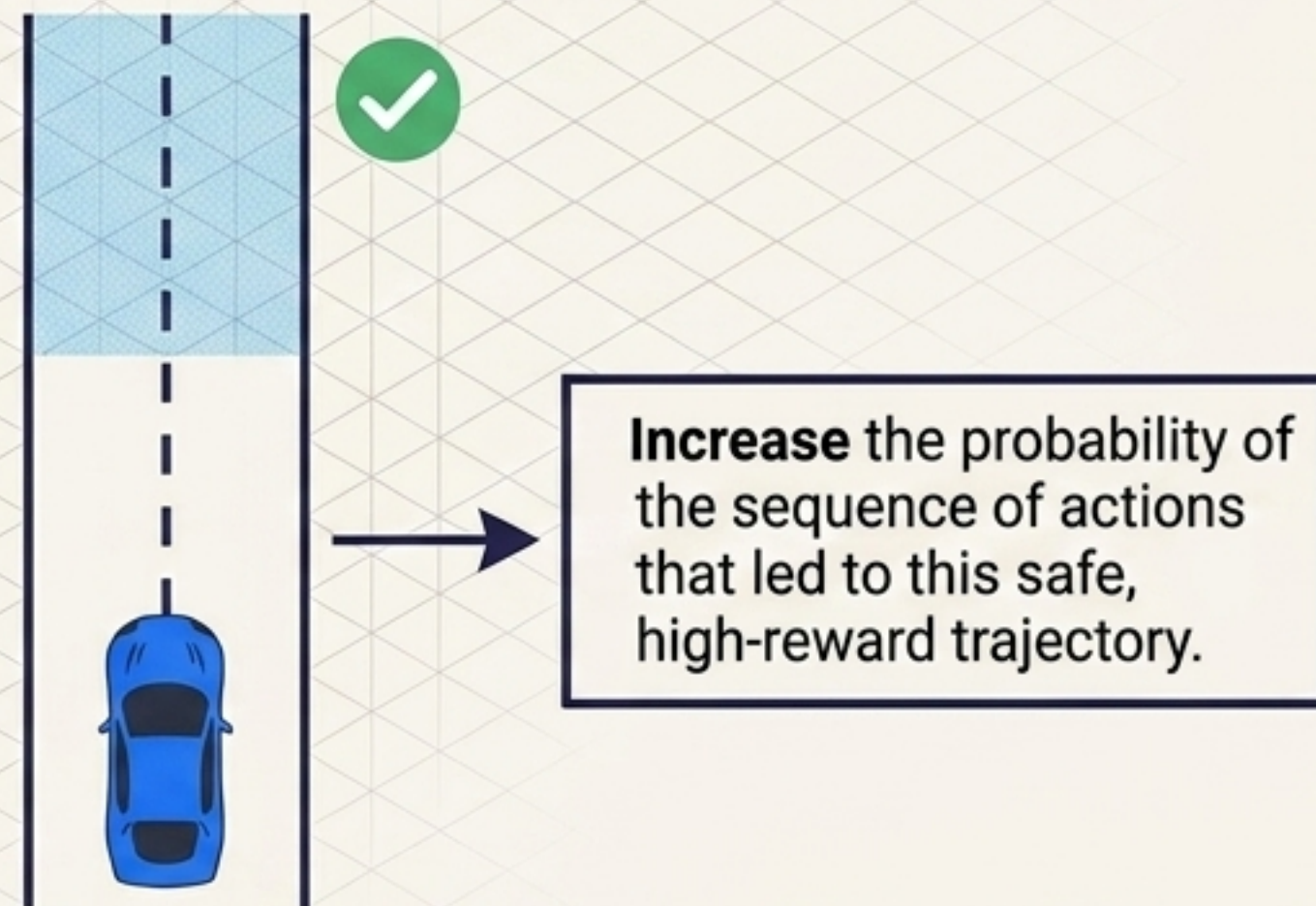


The Policy Gradient Update: Reinforcing Success

Scenario A: The Crash (Low Reward)



Scenario B: Success (High Reward)



The Policy Update

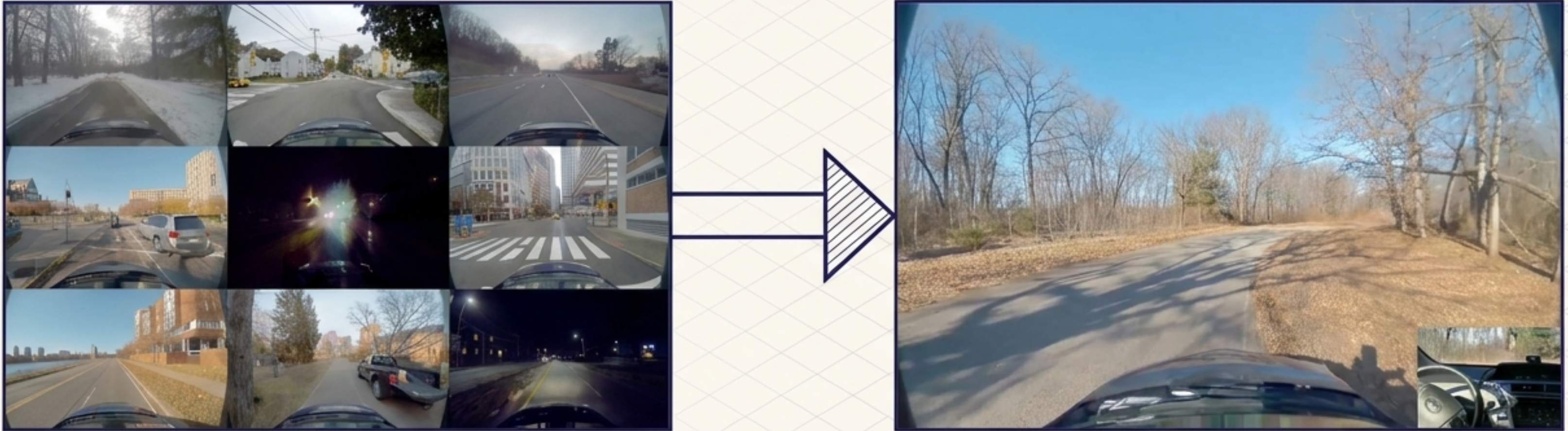
$$w' = w + \nabla \log P(a_t | s_t) R_t$$

Mathematically nudge the network's weights in the direction of higher rewards.

The Master Synthesis

Value-Based (e.g., DQN)	Policy-Based (e.g., Policy Gradients)
Action-value function $Q(s,a)$. Learns exactly how good each action is.	The policy $\pi(s)$ directly. Learns exactly how to choose actions.
Output Layer: One discrete Q-value per possible action.	Probability parameters (like Mean μ and Variance σ^2).
Action Selection: (takes the action with the maximum Q-value).	Stochastic (fluidly sampled from the continuous distribution).
Best For: Discrete action spaces (Chess, Atari, Grid worlds).	Continuous action spaces (Robotics, Autonomous Driving).

Real-World Deployment: Autonomous Vehicles



Agent: The autonomous vehicle.	State: Continuous camera and LIDAR data.	Action: Continuous steering wheel angle.	Reward: Distance travelled without crashing.
--	--	--	--

Breakthrough: A Policy Gradient RL agent trained entirely within a data-driven, high-fidelity simulator successfully deployed directly into the real world.

AlphaGo's Architectural Synthesis



The Ultimate Synthesis: AlphaGo achieved super-human performance not by choosing between Value and Policy, but by **architecturally** fusing them together.

Summary & Key Takeaways

01

The Engine of Autonomy

RL is uniquely suited for dynamic environments. Agents learn strictly through an active feedback loop: taking actions, observing new states, and maximising continuous future rewards.

02

The Two Paths of Intelligence

Value-based learning evaluates the definitive worth of every possible discrete move. Policy-based learning directly optimises the probability of actions, unlocking fluid, continuous control.

03

The Future is Hybrid

The most advanced and capable autonomous systems in the world combine robust value estimation with precise policy optimisation to achieve super-human capabilities.

Deep Reinforcement Learning transforms AI from passive pattern-recognisers into active problem-solvers.